

Finite Element Methods

Theory, Geometry, and Computation

Aldebert Lecornu

École Polytechnique de Louvain
Université catholique de Louvain

March 3, 2026

Contents

1	The diffusion equation: space and time	2
1.1	Strong formulation of the diffusion problem with one space dimension	2
1.1.1	Semi-discretization	3
1.2	Interpretation as a system of ODEs	4
1.2.1	The θ -scheme.	5
1.2.2	L^2 -stability when $\mathbf{f}(t) = 0$	5
1.2.3	Conservation of energy	6
1.2.4	Modal Analysis	7
1.3	Examples in Python	10
1.3.1	Model problem	11
1.3.2	L^2 projection of $u_0(x)$ in U_h	11
1.3.3	Time stepping	13
1.3.4	Modal Analysis	15
1.3.5	Why finite elements are too stiff	17
1.3.6	Why eigenvalues converge as h^{2k+2}	19

Chapter 1

The diffusion equation: space and time

We now turn to the diffusion equation in one spatial dimension and one time dimension. In contrast with the Poisson problem, which is stationary, diffusion is an intrinsically time-dependent process. In this chapter, we first adopt a classical approach based on a *semi-discretization*: the problem is discretized in space using finite elements, while time remains continuous. This leads to a system of ordinary differential equations, which can then be advanced in time using standard time-integration schemes. We conclude this chapter by moving beyond the semi-discrete formulation. Rather than discretizing space first and treating time separately, we introduce a finite-element discretization in time. We will see that specific choices of trial and test spaces in time recover classical time-stepping methods, including the θ -scheme.

1.1 Strong formulation of the diffusion problem with one space dimension

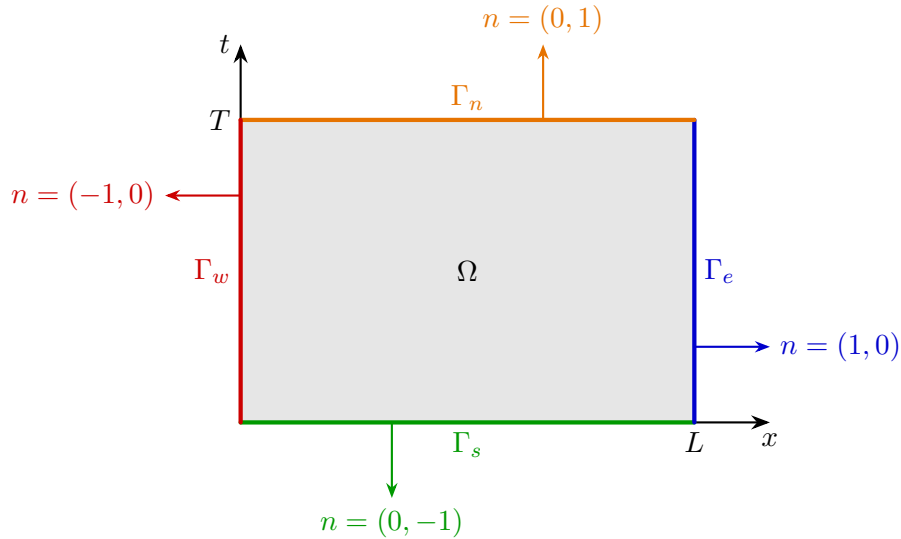


Figure 1.1: Space-time domain Ω with boundary parts $\Gamma_{n,s,e,w}$ and outward unit normals $n = (n_x, n_t)$.

The space-time domain is the rectangle $\Omega = [0, L] \times [0, T]$, whose boundary $\partial\Omega = \Gamma_w \cup \Gamma_e \cup \Gamma_s \cup \Gamma_n$ is decomposed into the west ($x = 0$), east ($x = L$), south ($t = 0$), and north ($t = T$) sides, with outward unit normals respectively $n = (-1, 0)$, $n = (1, 0)$, $n = (0, -1)$, and $n = (0, 1)$. We consider the diffusion equation: find $u(x, t)$ solution of

$$\partial_t u(x, t) - \partial_x (\kappa(x, t) \partial_x u(x, t)) = f(x, t) \quad \text{in } \Omega \quad (1.1)$$

supplemented with boundary conditions

$$u(x, t) = \bar{u}(t) \quad \text{on } \Gamma_w, \quad \kappa \partial_x u(x, t) n_x = \bar{q}(t) \quad \text{on } \Gamma_e,$$

where n is the normal vector to Γ_N and an initial condition

$$u(x, 0) = u_0(x) \quad \text{on } \Gamma_s.$$

For a time-oriented problem like diffusion, boundary data can only be prescribed on the parts of $\partial\Omega$ where the outward normal has a negative time component i.e. points toward the past. Mathematically if $n = (n_x, n_t)$ then one can impose boundary conditions when $n_t \leq 0$. In the present example we impose Dirichlet conditions on the west boundary and Neumann conditions on the east boundary, but this is merely a modeling choice: one could equally impose two Dirichlet conditions, two Neumann conditions, or exchange the Dirichlet and Neumann boundaries.

On the south boundary ($t = 0$) we impose a Dirichlet condition, which is in fact the only admissible choice: on the north boundary ($t = T$) the outward normal points toward the future and therefore no data can be prescribed, and moreover the diffusion equation contains only a first-order time derivative so that prescribing anything other than the value of u in time would overconstrain the problem.

1.1.1 Semi-discretization

Starting from the strong form of the diffusion equation (1.1), we multiply by a test function $\delta u(x) \in U_0 = \{\delta u(x) \in H^1([0, L]) \text{ , } \delta u(0) = 0\}$ and integrate over x :

$$\int_0^L \partial_t u \delta u \, dx - \int_0^L \partial_x (\kappa \partial_x u) \delta u \, dx = \int_0^L f \delta u \, dx.$$

Integrating the second term by parts gives

$$\int_0^L \partial_t u \delta u \, dx + \int_0^L \kappa \partial_x u \partial_x \delta u \, dx = \int_0^L f \delta u \, dx + \underbrace{\kappa \partial_x u(L, t) \delta u(L)}_{\bar{q}(t)} - \underbrace{\kappa \partial_x u(0, t) \delta u(0)}_{=0}.$$

Let $U_h \subset U$ be a finite element space, for instance the space of continuous P^k functions on a mesh of $[0, L]$, and let $\{\varphi_i\}_{i=1}^N$ be a basis of U_h . We seek an approximate solution of the form

$$u(x, t) \approx u_h(x, t) = \sum_{j=1}^N u_j(t) \varphi_j(x),$$

where the coefficients $u_j(t)$ are functions that depend on time. This is the standard choice of using finite elements only in space and leaving time continuous for now.

Choosing test functions/variations $\delta u_h = \varphi_i$ and inserting u_h into the weak formulation leads to the semi-discrete system

$$\sum_{j=1}^N \underbrace{\int_0^L \varphi_j \varphi_i \, dx}_{M_{ij}} \dot{u}_j(t) + \sum_{j=1}^N \underbrace{\int_0^L \kappa \partial_x \varphi_j \partial_x \varphi_i \, dx}_{K_{ij}} u_j(t) = f_i(t),$$

with

$$f_i(t) = \int_0^L f(x, t) \varphi_i(x) \, dx + \bar{q}(t) \varphi_i(L).$$

This result is obtained in exactly the same way as for the Poisson problem – the only new feature is the first term, which involves the mass matrix M multiplying the time derivative $\dot{u}_j(t)$. The following code shows how to compute the mass matrix with `gmsh` API.

```

import numpy as np
from scipy.sparse import lil_matrix

def assemble_M(elemType, elemTags, conn, jac, det, uvw, w, N):

    ne = len(elemTags)
    nloc = int(len(conn)/ne)
    ngp = len(w)
    nn = int(np.max(conn))

    # - Reshape data for making things easy
    det = np.array(det, dtype=np.float64).reshape(ne, ngp)
    jac = np.array(jac, dtype=np.float64).reshape(ne, ngp, 3, 3)
    conn = np.array(conn, dtype=np.int64).reshape(ne, nloc)
    N = np.array(N, dtype=np.float64).reshape(ngp, nloc)

    # Allocate global mass matrix
    M = lil_matrix((nn, nn), dtype=np.float64)

    # Assembly loops (no vectorization)
    for e in range(ne):
        nodes = conn[e, :] - 1 # global dof ids on element e

        for g in range(ngp):
            wg = w[g]
            detg = det[e, g]

            for a in range(nloc):
                Ia = int(nodes[a])
                Na = N[g, a]
                for b in range(nloc):
                    Ib = int(nodes[b])
                    M[Ia, Ib] += wg * Na * N[g, b] * detg

    return M

```

1.2 Interpretation as a system of ODEs

In matrix form, this system reads

$$M \dot{\mathbf{u}}(t) + K \mathbf{u}(t) = \mathbf{f}(t),$$

where M is the mass matrix, K is the Laplace matrix previously encountered in the Poisson problem, $\mathbf{u}(t) = (u_1(t), \dots, u_N(t))$ is the discrete vector of time-dependent degrees of freedom and $\mathbf{f}(t) = (f_1(t), \dots, f_N(t))$ is the time dependant right hand side.

The semi-discrete diffusion problem is a system of N coupled ordinary differential equations in time. The mass matrix M is symmetric positive definite, and the stiffness matrix K is symmetric positive semi-definite (positive definite once Dirichlet conditions are imposed). This structure reflects the dissipative nature of diffusion.

At this stage, time discretization has not yet been specified. Standard choices include explicit or implicit Euler schemes, Crank–Nicolson, or higher-order Runge–Kutta methods. The choice of time integrator is governed by stability and accuracy considerations and will be discussed separately.

1.2.1 The θ -scheme.

The idea now is to discretize the time variable, not by using a continuous approximation, but by employing standard finite differences. To this end, we partition the interval $[0, T]$ into discrete time levels

$$t_0 = 0, t_1, t_2, \dots, t_M = T.$$

Assume for simplicity a constant time step $\Delta t = t_{i+1} - t_i$. Let $t_n = n\Delta t$ and $\mathbf{u}^n \approx \mathbf{u}(t_n)$. The θ -scheme is defined by

$$M \frac{\mathbf{u}^{n+1} - \mathbf{u}^n}{\Delta t} + K \underbrace{(\theta \mathbf{u}^{n+1} + (1 - \theta) \mathbf{u}^n)}_{\mathbf{u}^{n+\theta}} = \theta \mathbf{f}^{n+1} + (1 - \theta) \mathbf{f}^n, \quad (1.2)$$

with $\mathbf{f}^n = \mathbf{f}(t_n)$. Rearranging yields the linear system

$$(M + \theta \Delta t K) \mathbf{u}^{n+1} = (M - (1 - \theta) \Delta t K) \mathbf{u}^n + \Delta t (\theta \mathbf{f}^{n+1} + (1 - \theta) \mathbf{f}^n). \quad (1.3)$$

The method advances the solution in time using increments of Δt : at each step, $\mathbf{u}^{n+1}$ is computed solely from the known state $\mathbf{u}^n$. This therefore constitutes a one-step time-integration scheme. The following code computes one time step using the θ -scheme.

```
def theta_step(M, K, F_n, F_np1, U_n, dt, theta, dir_dofs, dir_vals_n, dir_vals_np1):
    """
    One step of the theta-method for:
        M (U^{n+1} - U^n) / dt + K (theta U^{n+1} + (1 - theta) U^n) =
        theta F^{n+1} + (1 - theta) F^n

    Dirichlet BCs are imposed by reduction on the left system (at time n+1).
    We allow time-dependent Dirichlet values by using dir_vals_np1 in the reduction.
    """
    A = M + theta * dt * K
    B = M - (1.0 - theta) * dt * K

    rhs = B.dot(U_n) + dt * (theta * F_np1 + (1.0 - theta) * F_n)

    # Impose Dirichlet on A U^{n+1} = rhs
    A_red, rhs_red, free_dofs, U_full = apply_dirichlet_by_reduction(
        A, rhs, dir_dofs, dir_vals_np1
    )

    U_free = spsolve(A_red.tocsr(), rhs_red)
    U_full[free_dofs] = U_free
    U_full[dirichlet_dofs] = dirichlet_values
    return U_full
```

1.2.2 L^2 -stability when $\mathbf{f}(t) = 0$

The idea of L^2 stability comes directly from the continuous diffusion equation without right hand side/source term,

$$\partial_t u - \nabla \cdot (\kappa \nabla u) = 0.$$

Multiplying the equation by u and integrating over the domain Ω gives

$$\int_{\Omega} u \partial_t u \, dx - \int_{\Omega} u \nabla \cdot (\kappa \nabla u) \, dx = 0.$$

Integrating the second term by parts yields

$$\frac{1}{2} \frac{d}{dt} \int_{\Omega} u^2 dx + \int_{\Omega} \kappa |\nabla u|^2 dx = \int_{\partial\Omega} \kappa \partial_n u u ds.$$

If no flux enters or leaves the domain (i.e. homogeneous Neumann boundary conditions $\kappa \partial_n u = 0$ on $\partial\Omega$), the boundary term vanishes and we obtain

$$\frac{d}{dt} \int_{\Omega} u^2 dx = -2 \int_{\Omega} \kappa |\nabla u|^2 dx \leq 0.$$

Hence the L^2 norm of the solution cannot increase in time: diffusion reduces L^2 norm. A time discretization is therefore called L^2 -stable if it reproduces this fundamental property at the discrete level. For that, let us re-write (1.2) as:

$$M \frac{\mathbf{u}^{n+1} - \mathbf{u}^n}{\Delta t} + K \mathbf{u}^{n+\theta} = 0, \quad \mathbf{u}^{n+\theta} := \theta \mathbf{u}^{n+1} + (1 - \theta) \mathbf{u}^n.$$

Multiplying equation with $\mathbf{u}^{n+\theta}$ and using symmetry of M gives

$$\left(\frac{\mathbf{u}^{n+1} - \mathbf{u}^n}{\Delta t} \right)^T M \mathbf{u}^{n+\theta} + (\mathbf{u}^{n+\theta})^T K \mathbf{u}^{n+\theta} = 0.$$

If A is a symmetric semi-definite matrix, we define the A -inner product by

$$(\mathbf{v}, \mathbf{w})_A := \mathbf{v}^T A \mathbf{w} = \mathbf{w}^T A \mathbf{v},$$

with associated norm

$$\|\mathbf{v}\|_M := \sqrt{(\mathbf{v}, \mathbf{v})_M} = \sqrt{\mathbf{v}^T M \mathbf{v}}.$$

and use the identity

$$(\mathbf{a} - \mathbf{b})^T M (\theta \mathbf{a} + (1 - \theta) \mathbf{b}) = \frac{1}{2} (\|\mathbf{a}\|_M^2 - \|\mathbf{b}\|_M^2) + \left(\theta - \frac{1}{2} \right) \|\mathbf{a} - \mathbf{b}\|_M^2,$$

with $\mathbf{a} = \mathbf{u}^{n+1}$ and $\mathbf{b} = \mathbf{u}^n$, to obtain the discrete energy balance

$$\boxed{\frac{1}{\Delta t} (\|\mathbf{u}^{n+1}\|_M^2 - \|\mathbf{u}^n\|_M^2) = -2 \left(\theta - \frac{1}{2} \right) \frac{1}{\Delta t} \|\mathbf{u}^{n+1} - \mathbf{u}^n\|_M^2 - 2 \|\mathbf{u}^{n+\theta}\|_K^2.}$$

This is the exact discrete analogue of the continuous identity

$$\frac{d}{dt} \|u(t)\|_{L^2(\Omega)}^2 = -2 \|\nabla u(t)\|_{L^2(\Omega)}^2 \quad (\text{no flux, no source}),$$

with an additional *numerical dissipation* term $2(\theta - \frac{1}{2})\Delta t^{-1} \|\mathbf{u}^{n+1} - \mathbf{u}^n\|_M^2$ that vanishes for Crank–Nicolson ($\theta = \frac{1}{2}$) and is strictly positive (stable) for $\theta > \frac{1}{2}$. Thus, a θ scheme is L^2 -stable if $\theta \geq \frac{1}{2}$. The Crank–Nicolson is *marginally stable*.

1.2.3 Conservation of energy

In the isolated case (no flux through the east and west boundaries),

$$\kappa \partial_x u n_x = 0 \quad \text{on } \Gamma_w \cup \Gamma_e,$$

the diffusion equation

$$\partial_t u - \partial_x (\kappa \partial_x u) = 0$$

implies conservation of the total energy (first principle). Indeed, integrating over $[0, L]$ gives

$$\frac{d}{dt} \int_0^L u(x, t) dx = \int_0^L \partial_t u dx = \int_0^L \partial_x (\kappa \partial_x u) dx = [\kappa \partial_x u]_0^L = 0.$$

Therefore,

$$\int_0^L u(x, t) dx = \int_0^L u(x, 0) dx \quad \forall t \geq 0,$$

i.e. the total amount of u in the domain is conserved¹. Assume homogeneous Neumann boundary conditions on the west and east boundaries, so that the continuous problem conserves the total mass

$$\frac{d}{dt} \int_0^L u(x, t) dx = 0.$$

Let $\mathbf{1}$ denote the vector whose entries are all equal to 1. In a conforming finite element discretization one has

$$K\mathbf{1} = \mathbf{0},$$

since the Laplacian of a constant function vanishes. Consider again the θ -scheme without source term

$$M \frac{\mathbf{u}^{n+1} - \mathbf{u}^n}{\Delta t} + K \mathbf{u}^{n+\theta} = 0, \quad \mathbf{u}^{n+\theta} = \theta \mathbf{u}^{n+1} + (1 - \theta) \mathbf{u}^n.$$

Taking the M -inner product with $\mathbf{1}$ gives

$$\mathbf{1}^T M \frac{\mathbf{u}^{n+1} - \mathbf{u}^n}{\Delta t} + \mathbf{1}^T K \mathbf{u}^{n+\theta} = 0.$$

Because $K\mathbf{1} = \mathbf{0}$ and K is symmetric,

$$\mathbf{1}^T K \mathbf{u}^{n+\theta} = (K\mathbf{1})^T \mathbf{u}^{n+\theta} = 0.$$

Hence

$$\mathbf{1}^T M \mathbf{u}^{n+1} = \mathbf{1}^T M \mathbf{u}^n.$$

Since $\mathbf{1}^T M \mathbf{u}$ is the discrete counterpart of $\int_0^L u(x, t) dx$, the θ -scheme exactly preserves the total energy:

$$\mathbf{1}^T M \mathbf{u}^n = \mathbf{1}^T M \mathbf{u}^0 \quad \forall n.$$

Thus the θ -scheme mimics the continuous conservation property (it must be the case for any consistent time stepping scheme).

1.2.4 Modal Analysis

We have just seen that the θ scheme preserves exactly the energy, if we define the energy as the spatial average of the variable over the domain (for an isolated system). This is a discrete counterpart of the first law of thermodynamics. We have also seen that the L^2 norm of u must decrease – this is in some sense the translation of the second law – information decreases.

In a very simple setting, consider minimizing $\sum_i x_i^2$ under the constraint that $\sum_i x_i$ is fixed. The optimal solution is obviously that all x_i are equal, i.e. the state becomes constant.

This is exactly what happens in an isolated system governed by the diffusion equation: the total energy is conserved while the L^2 norm decreases, forcing the solution to converge irreversibly toward a constant state.

¹If u is a temperature and heat capacity is constant, the integral of the temperature is the thermal energy that is conserved in an isolated system.

However, one may ask how fast this convergence occurs. The two previous results do not provide any quantitative information about the rate of decay. To address this question, we now introduce the notion of modes.

The eigenfunctions $\phi_n(x)$ of ∂_{xx} with Neumann boundary conditions are solution of

$$\partial_{xx}\phi_n + \lambda_n\phi_n = 0 \quad \partial_x\phi_n(0) = \partial_x\phi_n(L) = 0.$$

We find

$$\phi_n(x) = \cos\left(\frac{n\pi}{L}x\right), \quad n \geq 0,$$

with eigenvalues

$$\lambda_n = \left(\frac{n\pi}{L}\right)^2, \quad n \geq 0.$$

Therefore the exact solution can be expanded as

$$u(x, t) = \sum_{n=0}^{\infty} a_n \cos\left(\frac{n\pi}{L}x\right) \exp\left(-\kappa \left(\frac{n\pi}{L}\right)^2 t\right),$$

where the coefficients a_n are determined by the initial condition $u(x, 0)$. Each cosine mode

$$\cos\left(\frac{n\pi}{L}x\right)$$

has wavenumber

$$k_n = \frac{n\pi}{L}.$$

Its (spatial) wavelength is therefore

$$\boxed{\lambda_n = \frac{2\pi}{k_n} = \frac{2L}{n}} \quad (n \geq 1),$$

and its spatial period is exactly the same quantity (one full oscillation):

$$\boxed{T_n^{\text{space}} = \lambda_n = \frac{2L}{n}}.$$

The temporal decay of the mode is governed by

$$\exp(-\kappa k_n^2 t) = \exp\left(-\kappa \left(\frac{n\pi}{L}\right)^2 t\right),$$

so the characteristic decay time is

$$\boxed{\tau_n = \frac{1}{\kappa k_n^2} = \frac{L^2}{\kappa n^2 \pi^2}}.$$

Hence, the smaller the wavelength λ_n (i.e. the larger n), the larger k_n^2 and the faster the exponential damping: *short-wavelength modes are dissipated first, while long-wavelength modes persist much longer.*

Consider the semi-discrete diffusion problem

$$M\dot{\mathbf{u}}(t) + K\mathbf{u}(t) = \mathbf{0}.$$

We assume that the discrete solution at time t_n , $\mathbf{u}(t_n) = \mathbf{u}^n$ is expanded into discrete modes:

$$\mathbf{u}^n = \sum_i a_i^n \phi_i.$$

where the modes ϕ_i are solutions of the generalized eigenvalue problem:

$$K\phi_i = \lambda_i M\phi_i, \quad 0 = \lambda_1 \leq \lambda_2 \leq \dots,$$

The θ scheme writes

$$(M + \theta\Delta t K)\mathbf{u}^{n+1} = (M - (1 - \theta)\Delta t K)\mathbf{u}^n.$$

Introducing the expansion into modes and using the eigen-relation $K\phi_i = \lambda_i M\phi_i$, we obtain

$$(M + \theta\Delta t K) \sum_i a_i^{n+1} \phi_i = (M - (1 - \theta)\Delta t K) \sum_i a_i^n \phi_i.$$

Since

$$(M + \theta\Delta t K)\phi_i = (1 + \theta\Delta t \lambda_i)M\phi_i, \quad (M - (1 - \theta)\Delta t K)\phi_i = (1 - (1 - \theta)\Delta t \lambda_i)M\phi_i,$$

the scheme decouples mode-by-mode:

$$(1 + \theta\Delta t \lambda_i) a_i^{n+1} M\phi_i = (1 - (1 - \theta)\Delta t \lambda_i) a_i^n M\phi_i.$$

Therefore,

$$a_i^{n+1} = g_\theta(\lambda_i) a_i^n, \quad g_\theta(\lambda) = \frac{1 - (1 - \theta)\Delta t \lambda}{1 + \theta\Delta t \lambda}.$$

The discrete modes are meant to approximate the continuous Fourier modes. In the continuous problem, each spatial mode

$$\cos\left(\frac{i\pi}{L}x\right)$$

is an eigenfunction of the Laplacian with eigenvalue

$$\lambda_i = \kappa \left(\frac{i\pi}{L}\right)^2,$$

and its amplitude evolves as

$$a_i(t_{n+1}) = a_i(t_n) e^{-\lambda_i \Delta t}.$$

Two natural questions arise. First, are the continuous eigenmodes $\cos(\frac{i\pi}{L}x)$ well approximated by the discrete modes ϕ_i ? The answer is yes: the generalized eigenvectors of

$$K\phi_i = \lambda_i M\phi_i$$

constitute a discrete analogue of the Laplacian eigenfunctions. As the mesh is refined, each discrete mode converges to the corresponding continuous Fourier mode, and the discrete eigenvalues λ_i converge to the exact diffusion rates $\kappa \left(\frac{i\pi}{L}\right)^2$. In other words, the spatial discretization correctly reproduces the wavelengths and their associated decay rates.

Second, does the θ -scheme reproduce the exponential decay? For the continuous problem, each mode evolves according to

$$a_i(t_{n+1}) = e^{-\lambda_i \Delta t} a_i(t_n).$$

The time discretization replaces this exact factor by

$$a_i^{n+1} = \frac{1 - (1 - \theta)\Delta t \lambda}{1 + \theta\Delta t \lambda} a_i^n.$$

To compare the θ -scheme with the exact exponential decay, set $z = \lambda\Delta t$ and write

$$g_\theta(z) = \frac{1 - (1 - \theta)z}{1 + \theta z}.$$

Using the geometric series expansion

$$\frac{1}{1 + \theta z} = 1 - \theta z + \theta^2 z^2 - \theta^3 z^3 + O(z^4), \quad (|z| \text{ small}),$$

we obtain

$$g_\theta(z) = (1 - (1 - \theta)z)(1 - \theta z + \theta^2 z^2 - \theta^3 z^3 + O(z^4)).$$

Multiplying and collecting terms yields

$$g_\theta(z) = 1 - z + \theta z^2 - \theta^2 z^3 + O(z^4).$$

In terms of λ and Δt ,

$$g_\theta(\lambda) = 1 - \lambda\Delta t + \theta \lambda^2 \Delta t^2 - \theta^2 \lambda^3 \Delta t^3 + O(\Delta t^4).$$

For comparison,

$$e^{-z} = 1 - z + \frac{z^2}{2} - \frac{z^3}{6} + O(z^4).$$

Hence the θ -scheme matches the exponential at first order for any θ , and it matches the quadratic term if and only if $\theta = \frac{1}{2}$.

1.3 Examples in Python

We have established a number of theoretical results for the diffusion equation in one spatial dimension using a time semi-discretization. In particular, we showed that the spatial part of the diffusion operator can be reused almost entirely from the chapter devoted to the one-dimensional Poisson equation discretized by the finite element method.

The only new ingredient is the *mass matrix*. While the stiffness matrix K represents the discrete Laplacian operator and therefore encodes spatial interactions (diffusive fluxes between neighbouring degrees of freedom), the mass matrix M has a different meaning: it represents the discrete L^2 inner product,

$$(u, v)_M = u^T M v \approx \int_{\Omega} u(x)v(x) dx.$$

In other words, M measures how much “quantity” is stored at each node. Physically, it plays the role of inertia or thermal capacity: the diffusion process does not instantly react to gradients, it first accumulates or releases content locally, and this storage effect is precisely what the mass matrix models.

Using the stiffness matrix K and the mass matrix M , the diffusion equation can therefore be written in semi-discrete form, namely as a system of ordinary differential equations

$$M\dot{u}(t) + Ku(t) = f(t).$$

We then performed a complete analysis of the time discretization, the so-called θ -scheme, and proved its fundamental properties, in particular conservation properties and stability behavior.

We now turn to the numerical verification of these theoretical results. For this purpose, we will use the small Python codes provided in the text. They will allow us to observe, in practice, the behavior predicted by the analysis.

1.3.1 Model problem

We consider the 1D diffusion equation

$$\partial_t u - \kappa \partial_{xx} u = 0 \quad \text{in } [0, L] \times [0, \infty[,$$

with homogeneous Neumann (insulated) boundary conditions

$$\partial_x u(0, t) = 0, \quad \partial_x u(L, t) = 0 \quad \forall t > 0,$$

a constant thermal conductivity $\kappa > 0$ and a given initial temperature $u(x, 0) = u_0(x)$. Using standard separation of variables, we easily get the general form of the space time solution

$$u(x, t) = a_0 + \sum_{m=1}^{\infty} a_m \cos\left(\frac{m\pi x}{L}\right) \exp\left(-\kappa\left(\frac{m\pi}{L}\right)^2 t\right),$$

where the coefficients are

$$a_0 = \frac{1}{L} \int_0^L u_0(x) dx, \quad a_m = \frac{2}{L} \int_0^L u_0(x) \cos\left(\frac{m\pi x}{L}\right) dx \quad (m \geq 1).$$

Conservation of mass. Integrating the PDE over $(0, L)$ and using the Neumann boundary conditions yields

$$\frac{d}{dt} \int_0^L u(x, t) dx = \kappa \partial_x u(L, t) - \kappa \partial_x u(0, t) = 0,$$

hence the total heat content $\int_0^L u(x, t) dx$ is conserved and equals $L a_0$.

Decay of L^2 energy. Define $E(t) = \int_0^L u(x, t)^2 dx$, we already proved that

$$\frac{dE}{dt} = -2\kappa \int_0^L (\partial_x u)^2 dx \leq 0.$$

From our analytical solution above and using orthogonality, we get

$$E(t) = L a_0^2 + \frac{L}{2} \sum_{m=1}^{\infty} a_m^2 \exp\left(-2\kappa\left(\frac{m\pi}{L}\right)^2 t\right).$$

A multiscale initial condition. A convenient multiscale choice is a finite cosine series:

$$u_0(x) = a_0 + \sum_{m \in \mathcal{M}} A_m \cos\left(\frac{m\pi x}{L}\right), \quad \mathcal{M} = \{1, 3, 5, 7\},$$

so that the exact solution is obtained by simply damping each mode in time.

1.3.2 L^2 projection of $u_0(x)$ in U_h

The continuous initial condition $u_0(x)$ is, in general, not contained in the finite element space U_h . Since our numerical method evolves a vector of degrees of freedom, we must first construct a discrete function $u_h^0 \in U_h$ that represents u_0 as faithfully as possible.

The natural choice is the L^2 projection. We define u_h^0 as the function in U_h that is closest to u_0 in the L^2 sense, i.e.

$$u_h^0 = \arg \min_{v_h \in U_h} \int_{\Omega} (v_h(x) - u_0(x))^2 dx.$$

Taking the first variation with respect to v_h gives the orthogonality condition

$$\int_{\Omega} (u_h^0 - u_0) \delta u_h^0 dx = 0 \quad \forall \delta u_h^0 \in U_h,$$

which means that the projection error is orthogonal to the finite element space. Let $\{\varphi_i\}_{i=1}^n$ be the basis of U_h and write

$$u_h^0(x) = \sum_{j=1}^n u_j^0 \varphi_j(x).$$

Choosing successively $\delta u_h^0 = \varphi_i$ yields the linear system

$$\sum_{j=1}^n \underbrace{\int_{\Omega} \varphi_i(x) \varphi_j(x) dx}_{M_{ij}} u_j^0 = \underbrace{\int_{\Omega} u_0(x) \varphi_i(x) dx}_{b_i}.$$

Finding u_j^0 consist again in solving a linear system that involves our well known *mass matrix* M and a right-hand side vector whose entries b_i are the moments of the continuous initial condition u_0 against the basis functions φ_i . The following code does the computation of the b_i 's.

```
def l2proj(elemTags, conn, det, xphys, w, N, nodeCoords, u0_fun):

    ne = len(elemTags)
    ngp = len(w)
    nloc = int(len(conn) // ne)
    nn = int(np.max(conn))
    b = np.zeros(nn, dtype=float)

    det = np.asarray(det, dtype=np.float64).reshape(ne, ngp)
    conn = np.asarray(conn, dtype=np.int64).reshape(ne, nloc)
    N = np.asarray(N, dtype=np.float64).reshape(ngp, nloc)
    xphys = np.asarray(xphys, dtype=np.float64).reshape(ne, ngp, 3)

    for e in range(ne):
        nodes = conn[e, :] - 1
        for g in range(ngp):
            xg = xphys[e, g]
            wg = w[g]
            detg = det[e, g]
            u0g = u0_fun(xg)
            for a in range(nloc):
                Ia = int(nodes[a])
                print(u0g)
                b[Ia] += u0g * N[g, a] * detg * wg
    return b
```

Let us consider a simple example. We take the interval $[0, 1]$ and discretize it with 5 finite elements of polynomial degree k . We choose the multiscale initial condition

$$u_0(x) = 1 + \cos(6\pi x) + \cos(16\pi x).$$

This function contains three distinct spatial scales:

- the constant mode,
- a medium frequency mode $\cos(6\pi x)$,

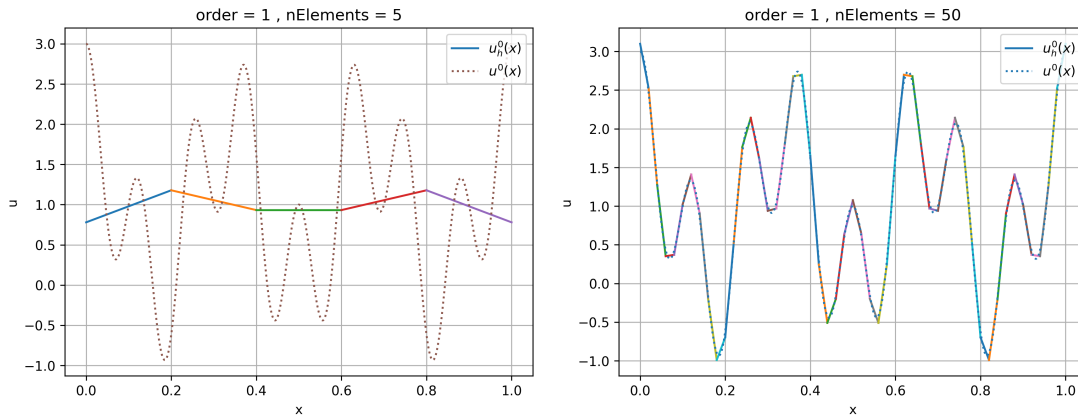


Figure 1.2: L^2 projection of $u_0(x) = 1 + \cos(6\pi x) + \cos(16\pi x)$ on a 1D mesh of 5 P^1 elements (left) and 50 P^1 elements (right).

- a higher frequency mode $\cos(16\pi x)$.

Since $\cos(6\pi x)$ corresponds to 3 full oscillations over $[0, 1]$ and $\cos(16\pi x)$ to 8 oscillations, it is clear that the ability of the finite element space U_h to represent these components depends strongly on the polynomial degree k and on the mesh resolution.

With only 5 elements over $[0, 1]$, the mesh size is $h = 0.2$. If low-order elements (e.g. $k = 1$) are used, the discrete space has limited approximation power. The constant mode will be represented exactly, but the oscillatory components will be poorly resolved (see Figure 1.2, left). On the other hand, with 50 P^1 elements (see Figure 1.2, right), we clearly see that the finite element space becomes rich enough to represent at least the medium-frequency mode accurately. Indeed, the mesh is now fine enough to provide several degrees of freedom per wavelength of $\cos(6\pi x)$, so the L^2 projection captures this component with the correct phase and amplitude. The highest-frequency mode $\cos(16\pi x)$ is still more demanding and remains less accurately represented, but the improvement brought by the finer discretization is already evident on the projected initial condition.

As k increases, each element can represent higher-degree polynomials, which improves the approximation of oscillatory behavior locally. For sufficiently large k , the finite element space becomes rich enough to capture both cosine modes with good accuracy, and the L^2 projection error decreases significantly. For example, Figure 1.3 shows the same comparison as Figure 1.2, but using 5 elements of order 3 (left) and of order 6 (right).

1.3.3 Time stepping

Let us now evolve the initial condition

$$u_0(x) = 1 + \cos(6\pi x) + \cos(16\pi x)$$

with the diffusion operator on $[0, 1]$ with homogeneous Neumann (insulated) boundary conditions, and with diffusion coefficient $\kappa = 1$.

The constant part $a_0 = 1$ is an eigenfunction associated with the eigenvalue 0 of the Neumann Laplacian. It is therefore preserved exactly by the continuous diffusion equation, and it must also remain unchanged numerically. In particular, we have shown that any θ -scheme (and more generally any consistent time discretization) preserves this constant mode.

The two oscillatory components have the following wavelengths. A function $\cos(2\pi f x)$ has wavelength $\lambda = 1/f$ on $[0, 1]$. Hence

$$\cos(6\pi x) = \cos(2\pi \cdot 3 x) \quad \Rightarrow \quad \lambda_1 = \frac{1}{3}, \quad \cos(16\pi x) = \cos(2\pi \cdot 8 x) \quad \Rightarrow \quad \lambda_2 = \frac{1}{8}.$$

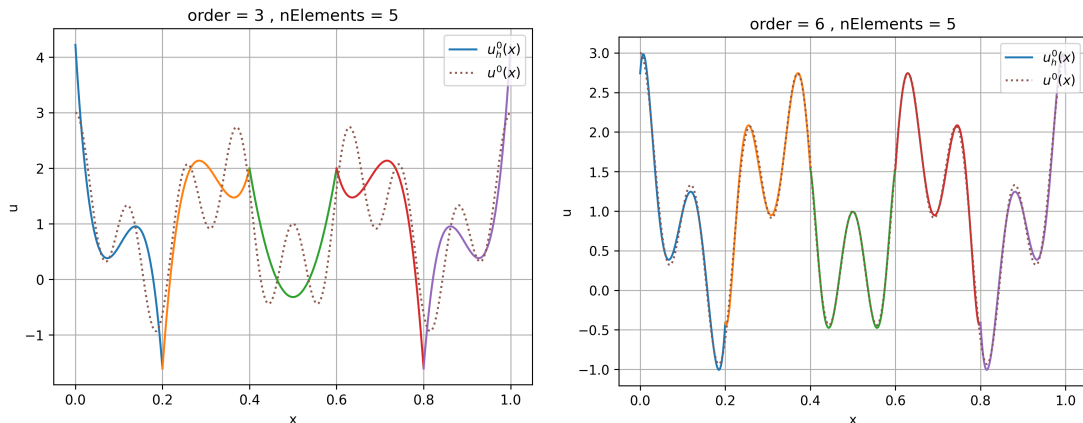


Figure 1.3: L^2 projection of $u_0(x) = 1 + \cos(6\pi x) + \cos(16\pi x)$ on a 1D mesh of 5 P^3 elements (left) and 5 P^6 elements (right).

Moreover, under diffusion

$$u_t - u_{xx} = 0,$$

each Neumann mode $\cos(m\pi x)$ decays exponentially like $\exp(-(m\pi)^2 t)$. In our case the two modes correspond to $m = 6$ and $m = 16$, so their amplitudes decay as

$$\exp(-(6\pi)^2 t) \quad \text{and} \quad \exp(-(16\pi)^2 t).$$

A convenient characteristic decay time is the e^{-1} time constant

$$\tau_m = \frac{1}{(m\pi)^2}.$$

Therefore,

$$\tau_1 = \frac{1}{(6\pi)^2} = \frac{1}{36\pi^2}, \quad \tau_2 = \frac{1}{(16\pi)^2} = \frac{1}{256\pi^2}.$$

Numerically,

$$\tau_1 \approx \frac{1}{36 \cdot 9.8696} \approx 2.81 \times 10^{-3}, \quad \tau_2 \approx \frac{1}{256 \cdot 9.8696} \approx 3.95 \times 10^{-4}.$$

In particular,

$$\frac{\tau_1}{\tau_2} = \frac{256}{36} \approx 7.11,$$

so the high-frequency component $\cos(16\pi x)$ decays about seven times faster than the medium-frequency component $\cos(6\pi x)$. By choosing a time step $\Delta t = 10^{-5}$, we perform roughly

$$\frac{\tau_2}{\Delta t} \approx \frac{3.95 \times 10^{-4}}{10^{-5}} \approx 40$$

time steps over the fastest characteristic decay time τ_2 . This is therefore a very fine temporal resolution of the rapid mode $\cos(16\pi x)$.

In such a regime, the time discretization error becomes comparatively small, since the transient is sampled by many time steps before its amplitude has significantly decreased. In contrast, the spatial resolution is limited by the mesh size and the polynomial degree: for a given wavelength, the finite element space can only represent oscillations up to a certain accuracy. Consequently, once Δt is sufficiently small, further refinement in time yields little improvement, and the global error is expected (and indeed observed) to be dominated by the spatial discretization error. Figure 1.4 shows the exact solution $u(x, T)$ and the finite element solution $u_h(x, T)$ for

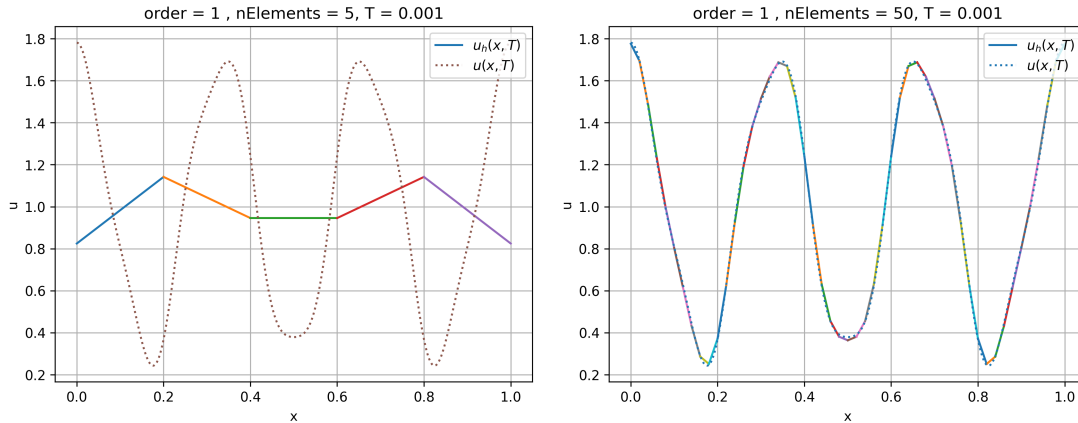


Figure 1.4: Exact solution $u(x, T)$ and numerical solution $u_h(x, T)$ for $T = 10^{-3}$. Both finite element solutions use P^1 elements, left Figure has 5 elements and right Figure 50.

$T = 10^{-3}$ i.e. when the fast mode is already gone but the slow one is still alive. Both simulations have been done using P^1 elements but the most refined one (with 50 elements) has about 17 elements per wavelength for the intermediary mode $\cos(6\pi x)$ which is considered to be resolved. Figure 1.4 shows that, at $T = 10^{-3}$, the finite element solution with 50 P^1 elements is very clean, way cleaner than at $T = 0$ (Figure 1.2). Unresolved high frequency modes have quickly been removed and resolved modes decay at the right rate. We will examine later the accuracy of numerical λ 's but at the first sight, modes that are discretized with about 17 P^1 elements are well resolved. Now let's see what happens at higher order. Figure 1.5 shows again the exact solution $u(x, T)$ and the finite element solution $u_h(x, T)$ for $T = 10^{-3}$ but using 5 high order elements ($k = 3$ and $k = 5$). In both cases we use only 5 elements, which yields slightly fewer than two elements per wavelength for the component $\cos(6\pi x)$ (since its wavelength is $\lambda_1 = 1/3$). As suggested by the figure, with cubic elements ($k = 3$) this level of spatial resolution is not entirely satisfactory: the oscillatory mode is captured, but not with full accuracy (see Figure 1.5, left). Nevertheless, at $T = 10^{-3}$ the numerical solution is already much closer to the analytical solution than the initial L^2 projection. This is expected, since diffusion rapidly damps the poorly resolved high-frequency content.

In the forthcoming modal analysis, we will make this observation more precise: the relevant numerical mode (i.e. the eigenvector of $M^{-1}K$) and its associated eigenvalue are reasonably close to their continuous counterparts. For $\cos(6\pi x)$, the continuous wavenumber is $f = 3$ (wavelength $\lambda_1 = 1/3$), and the corresponding continuous Laplacian eigenvalue is $(6\pi)^2$. While the discrete eigenpair is already a good approximation with $k = 3$, it is not yet perfect on such a coarse mesh. Unsurprisingly, when using higher-order elements (e.g. $k = 6$, Figure 1.5/right), the same mesh becomes sufficient to represent the mode essentially exactly, and the computed solution is then fully resolved.

In summary, when a spatial mode is well represented by the finite element space, its numerical evolution closely follows the analytical behavior. When a mode is poorly represented, it still appears to vanish rapidly and does not seem to pollute the solution for longer than expected. At this stage, this is only an empirical observation. Let us now examine this phenomenon more carefully.

1.3.4 Modal Analysis

The most interesting configuration in the previous section is the one using 5 elements of polynomial degree $k = 3$. We now turn to a modal point of view and compute the solution of the

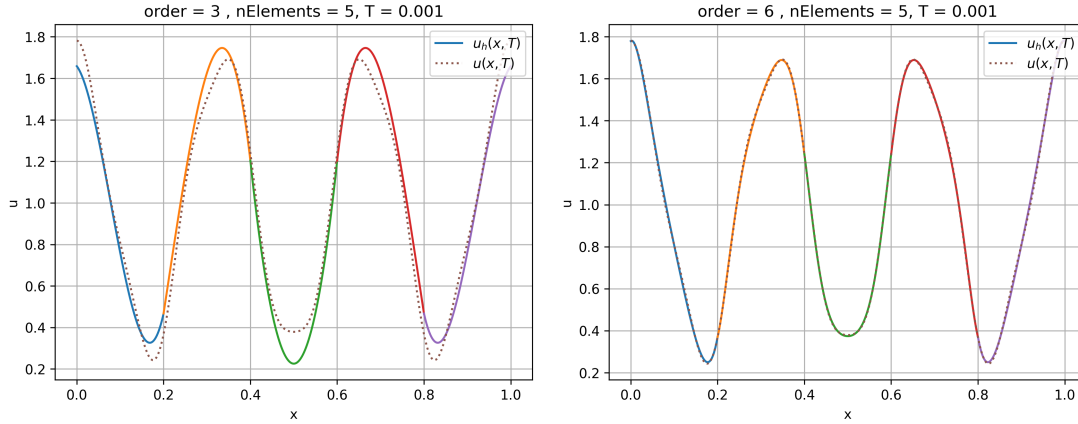


Figure 1.5: Exact solution $u(x, T)$ and numerical solution $u_h(x, T)$ for $T = 10^{-3}$ using 5 elements of order $k = 3$ (left) and $k = 5$ (right).

associated generalized eigenvalue problem

$$K v_i + \lambda_i M v_i = 0.$$

Equivalently, assuming $\lambda_i \neq 0$,

$$K v_i = \mu_i M v_i, \quad \text{with } \mu_i = -\lambda_i,$$

or, in operator form,

$$M^{-1} K v_i = \mu_i v_i.$$

In Python, we have K and M in CSR (compressed storage by row) format and it is possible say to compute efficiently the 10 smallest eigenvalues with their respective eigenvectors v_i .

```
import numpy as np
from scipy.sparse.linalg import eigsh

def neumann_exact_eigs(L, k):
    m = np.arange(k, dtype=float)
    return (m * np.pi / L)**2

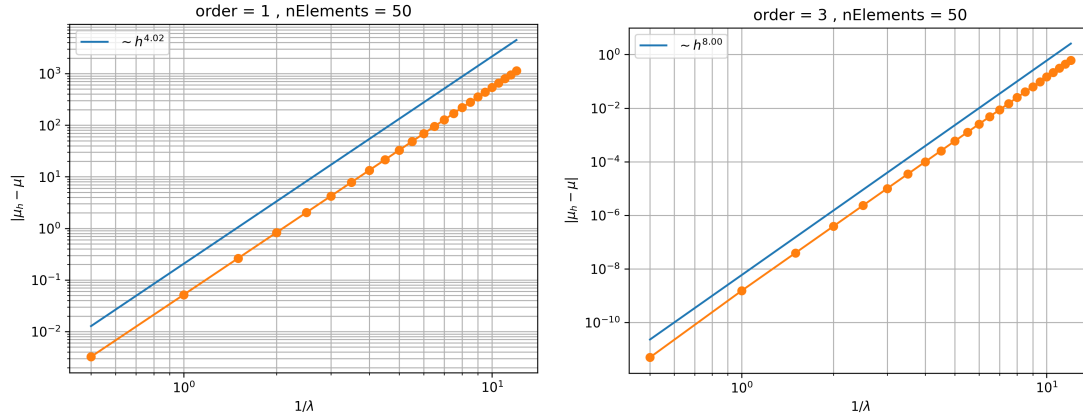
def smallest_generalized_eigs(K, M, k=10, L=1.0):
    mu, V = eigsh(K, k=k, M=M, sigma=0.0, which="LM")
    idx = np.argsort(mu)
    mu = mu[idx]
    V = V[:, idx]

    mu_ex = neumann_exact_eigs(L, k)

    print(" i      mu_FE                mu_exact                rel.err")
    for i in range(k):
        denom = max(abs(mu_ex[i]), 1e-30)
        rel = abs(mu[i] - mu_ex[i]) / denom
        print(f"{i:2d}  {mu[i]:.12e}  {mu_ex[i]:.12e}  {rel:.3e}")

    return mu, V, mu_ex
```

We have run the code with a mesh of 50 elements at orders $k = 1$ and $k = 3$. We observe that the convergence is of order $(1/\lambda)^{2k+2}$ which is twice the order of convergence of the L^2 error of the finite element solution – and so of the modes v_i as well. Figure 1.6 shows the result for the 25

Figure 1.6: Convergence of the eigenvalues of $M^{-1}K$.

first eigenvalues. Now assume that we work on a mesh with 50 elements. The modal analysis can say lots of things on the finite element convergence with respect to a "size". Given an eigenmode $v_i(x)$ and its finite element approximation $v_{ih}(x)$ – we know that $\|v_{ih}(x) - v_i(x)\|_{L^2} = \mathcal{O}h^{k+1}$. we know that

1.3.5 Why finite elements are too stiff

Rayleigh quotients. The Rayleigh quotient provides a scalar measure of the “energy per unit mass” associated with a function. For the continuous problem, it is defined for $0 \neq v \in H^1(0, L)$ by

$$\mathcal{R}(v) = \frac{a(v, v)}{m(v, v)} = \frac{\int_0^L |v'(x)|^2 dx}{\int_0^L |v(x)|^2 dx}.$$

The numerator measures the elastic (or diffusion) energy of v , while the denominator measures its L^2 norm. The quotient is invariant under multiplication by a nonzero scalar:

$$\mathcal{R}(\alpha v) = \mathcal{R}(v),$$

so it depends only on the “direction” of v in H^1 .

If u_k is an exact eigenfunction of the Laplace operator with homogeneous boundary conditions,

$$a(u_k, v) = \mu_k m(u_k, v) \quad \forall v \in H^1,$$

then taking $v = u_k$ immediately gives

$$\mathcal{R}(u_k) = \mu_k.$$

Thus the eigenvalues are precisely the stationary values of the Rayleigh quotient, and the smallest eigenvalue satisfies

$$\mu_0 = \min_{0 \neq v \in H^1} \mathcal{R}(v).$$

The discrete Rayleigh quotient is defined analogously for $0 \neq v_h \in V_h$:

$$\mathcal{R}_h(v_h) = \frac{a(v_h, v_h)}{m(v_h, v_h)}.$$

If $v_h = \sum_i V_i \varphi_i$, then

$$\mathcal{R}_h(v_h) = \frac{V^T K V}{V^T M V},$$

which coincides with the matrix Rayleigh quotient of the generalized eigenvalue problem

$$KV = \mu_h MV.$$

In particular, if V is a discrete eigenvector, then

$$\mathcal{R}_h(v_h) = \mu_h.$$

The fundamental variational characterization of eigenvalues is given by the min–max Courant–Fischer principle:

$$\mu_m = \min_{\substack{S \subset H^1 \\ \dim S = m+1}} \max_{0 \neq v \in S} \mathcal{R}(v), \quad \mu_{m,h} = \min_{\substack{S_h \subset V_h \\ \dim S_h = m+1}} \max_{0 \neq v_h \in S_h} \mathcal{R}_h(v_h). \quad (1.4)$$

To understand this formula, recall first that

$$\mu_0 = \min_{0 \neq v \in H^1} \mathcal{R}(v),$$

so the first eigenvalue is obtained by minimizing the energy under the normalization $m(v, v) = 1$. The corresponding minimizer is the first eigenfunction u_0 .

For the second eigenvalue, we must exclude this first direction. Indeed, if no constraint is imposed, the minimization would again return u_0 . The next eigenvalue is therefore obtained by minimizing $\mathcal{R}(v)$ under the orthogonality constraint

$$m(v, u_0) = 0.$$

More generally, the m -th eigenvalue is obtained by minimizing the Rayleigh quotient over functions that are m -orthogonal to the first m eigenfunctions:

$$m(v, u_j) = 0 \quad j = 0, \dots, m-1.$$

The min–max formulation (1.4) encodes exactly this idea in a coordinate-free way. Instead of writing orthogonality constraints explicitly, we proceed as follows:

- choose any subspace S of dimension $m+1$,
- compute the largest Rayleigh quotient inside S ,
- then select the subspace that makes this worst value as small as possible.

The optimal subspace turns out to be

$$S = \text{span}\{u_0, \dots, u_m\},$$

and the corresponding maximal value is precisely μ_m .

The discrete characterization is identical, with H^1 replaced by V_h . The discrete eigenvalues are therefore obtained by the same successive energy minimization, but restricted to the finite dimensional space V_h .

This interpretation immediately explains the fundamental inequality that explains why finite elements are "too stiff":

$$\mu_m \leq \mu_{m,h}.$$

Indeed, since $V_h \subset H^1$, the minimization in the discrete case is performed over a smaller collection of admissible subspaces. Restricting the admissible set can only increase the minimum.

1.3.6 Why eigenvalues converge as h^{2k+2}

Finally, this framework gives the standard strategy for eigenvalue error estimates. Let

$$W_m = \text{span}\{u_0, \dots, u_m\}.$$

If we can construct a discrete subspace

$$W_{m,h} \subset V_h$$

that approximates W_m well (for instance by interpolating each u_j), then inserting $S_h = W_{m,h}$ into the discrete min-max formula yields

$$\mu_{m,h} \leq \max_{0 \neq v_h \in W_{m,h}} \mathcal{R}_h(v_h).$$

Thus the discrete eigenvalue is controlled by how well the exact eigenspace is approximated in energy norm. This is the key mechanism behind the estimate

$$|\mu_{m,h} - \mu_m| \lesssim \|u_m - u_{m,h}\|_{H^1}^2.$$

Hence, although the finite element approximation of eigenfunctions converges with order $k+1$ in the H^1 norm, the associated eigenvalues converge with the *squared* order,

$$\boxed{|\mu_{m,h} - \mu_m| = \mathcal{O}(h^{2k+2})},$$

which is precisely the $2k+2$ rate observed in the numerical experiments.

Bibliography
